From Keys to Indices -- The Art of Mapping

Lecture 16

Data Structures and Algorithms

A New Unit Begins

Where We Have Been, Where We Are Going

Over the previous unit, we studied **trees** -- hierarchical structures that give us O(log n) search, insertion, and deletion. BSTs, AVL trees, Red-Black trees, B-Trees, and heaps each solved a different version of the same problem: organizing data for efficient operations.

But we still have not answered a simple question:

Can we search for a key in O(1) time -- constant time, regardless of how many elements we store?

This lecture introduces the idea that makes O(1) lookup possible: **hashing**. We will learn how hash functions work, how to design them, and why they sometimes fail.

This is the first lecture of a new unit covering **Hash Tables** and **Graphs** -- two of the most practically important topics in all of computer science.

Part 1: The Direct Access Problem

The Search Problem -- A Quick Recap

Let us revisit the core problem that has driven every data structure we have studied:

Given a collection of n elements, find any specific element as fast as possible.

Data Structure	Search Time	Why?
Unsorted Array	O(n)	Must check every element
Sorted Array	O(log n)	Binary search halves the space each step
BST (balanced)	O(log n)	Left-right decisions at each level
AVL / Red-Black	O(log n)	Guaranteed balanced height

O(log n) is impressive. Searching a billion items requires only about 30 comparisons. But there is still a nagging thought: arrays give us O(1) access by index. If we know an element is at position 42, we fetch it instantly: `arr[42]`.

What if we could somehow compute the index directly from the key itself?

The Direct Access Idea

Suppose you are building a student database for a class of 100 students, each with a roll number between 1 and 100.

You could create an array of size 101 and store each student's record at the index equal to their roll number:

```
Student roll number 42  -->  record stored at arr[42]
Student roll number 7   -->  record stored at arr[7]
Student roll number 95  -->  record stored at arr[95]
```

Search time: O(1). Just compute `arr[rollNumber]`. No comparisons, no traversals.

This is called **direct addressing** -- the key itself is the index. It is the fastest possible search.

But there is a problem...

What happens when the keys are not small, neat integers between 1 and 100?

When Direct Access Breaks Down

Consider storing records indexed by 10-digit Indian phone numbers:

```
Key: 9876543210  -->  store at arr[9876543210]
```

This requires an array of size **10 billion**. Even if you only store 50 contacts, you waste memory for 10 billion empty slots. This is absurd.

What about non-numeric keys?

```
Key: "Ramesh Kumar"  -->  store at arr[???]
```

A string is not an integer. We cannot use it as an array index at all.

The two problems:

1. **Keys can be astronomically large** -- we cannot allocate an array that big
2. **Keys can be non-numeric** -- strings, objects, composite values

We need a way to **convert arbitrary keys into small, valid array indices**. This is exactly what a **hash function** does.

The Hashing Idea

A **hash function** takes a key from a potentially huge universe and maps it to a small integer -- an index into a manageable array.

```
h: Universe of Keys  -->  {0, 1, 2, ..., m-1}
```

Where `m` is the size of our array (called the **hash table**).

Example: Phone number 9876543210 with table size m = 100:

- $h(9876543210) = 9876543210 \mod 100 = 10$
- Store the record at `arr[10]`

Now we only need an array of 100 slots, not 10 billion.

The result of the hash function is called the **hash value**, **hash code**, or simply the **hash**.

Analogy: Think of a library using the Dewey Decimal System. A book titled "Introduction to Algorithms" does not get shelved under "I" -- it gets assigned a number like 005.1, and that number tells you exactly which shelf to go to. You do not search every shelf; you go directly to the right one. A hash function works the same way for data.

Part 2: Designing Hash Functions

What Makes a Good Hash Function?

Not all hash functions are created equal. A good hash function satisfies these properties:

1. Deterministic

The same key must always produce the same hash. If $h(\text{"Ramesh"}) = 7$ today, it must be 7 tomorrow, next week, and forever. Otherwise, we would never find our data again.

2. Uniform Distribution

Keys should be spread as evenly as possible across all slots. If a function sends 80% of keys to slot 0 and 20% to everything else, slot 0 becomes a bottleneck.

3. Efficient to Compute

The hash function should run in O(1) -- ideally a few arithmetic operations. If computing the hash takes O(n), we lose the advantage of hashing.

4. Minimize Collisions

Different keys should produce different hash values as much as possible. (We will see shortly that perfect avoidance of collisions is mathematically impossible.)

A deliberately terrible hash function: $h(k) = 0$ for all keys. Every record goes to slot 0. Search degrades to O(n). The function is deterministic and efficient, but the distribution is catastrophically non-uniform.

Method 1: The Division Method

The simplest and most widely used hash function:

$$h(k) = k \mod m$$

Divide the key by the table size and take the remainder.

Example: Table size $m = 10$, keys = {23, 47, 81, 95, 60}

Key (k)	Computation	h(k)	Slot
23	23 mod 10	3	Slot 3
47	47 mod 10	7	Slot 7
81	81 mod 10	1	Slot 1
95	95 mod 10	5	Slot 5
60	60 mod 10	0	Slot 0

All five keys landed in different slots -- this is the ideal outcome.

The Division Method: Choosing m Carefully

The choice of m has a dramatic effect on the quality of the hash function.

Bad choices:

- **m = power of 2 (e.g., 128, 256, 1024):** If $m = 2^p$, then $k \mod m$ simply extracts the **lowest p bits** of the key. All information in the higher bits is ignored, leading to poor distribution.
- **m = 100 or 1000:** If keys are phone numbers or roll numbers, patterns in the last digits (like area codes or year of admission) cause many keys to cluster into the same few slots.

Good choices:

Use a prime number that is not close to any power of 2.

Table Size Needed	Bad Choice	Good Choice (Prime)
~100	100 or 128	97
~1000	1000 or 1024	997
~10,000	10,000 or 16,384	9973

Why prime? A prime modulus interacts with more bits of the key, distributing values more uniformly. It avoids the "resonance" effects that occur when the table size shares factors with the key patterns.

Method 2: The Multiplication Method

When you need better distribution than division alone, use the multiplication method:

$$h(k) = \lfloor m \cdot (k \cdot A \mod 1) \rfloor$$

Where $0 < A < 1$ is a carefully chosen constant, and "$\mod 1$" means "take the fractional part."

Step-by-step:

1. Multiply the key by A
2. Extract the fractional part (discard the integer part)
3. Multiply the fractional part by m
4. Take the floor (round down to an integer)

The magic constant:

Donald Knuth, in *The Art of Computer Programming*, recommends:

$$A \approx \frac{\sqrt{5} - 1}{2} \approx 0.6180339\ldots$$

This is the inverse of the golden ratio. It produces a particularly uniform distribution of hash values.

The Multiplication Method: Worked Example

Compute $h(k)$ for $k = 123456$, $A = 0.618$, $m = 16$:

```
Step 1: k * A = 123456 * 0.618 = 76,295.808

Step 2: Fractional part = 76295.808 - 76295 = 0.808

Step 3: m * fractional = 16 * 0.808 = 12.928

Step 4: Floor = 12

Therefore:  h(123456) = 12
```

Key advantage over the division method:

The multiplication method works well **regardless of the table size m**. Even powers of 2 are perfectly fine, because the fractional multiplication "scrambles" the bits of the key before the table size comes into play.

Feature	Division Method	Multiplication Method
Formula	$k \mod m$	$\lfloor m \cdot (kA \mod 1) \rfloor$
Choice of m	Must be prime, not near 2^p	Any value works, even 2^p
Simplicity	Very simple	Slightly more complex
Distribution	Depends heavily on m	Generally more uniform

Hashing Non-Integer Keys: Strings

So far, our hash functions assume the key is an integer. But in practice, keys are often **strings** -- names, email addresses, file paths, variable names.

We need to convert a string to an integer first, then hash that integer.

Attempt 1: Sum of character codes

Convert each character to its ASCII value and add them up:

```
h("abc") = 97 + 98 + 99 = 294
h("bca") = 98 + 99 + 97 = 294
h("cab") = 99 + 97 + 98 = 294
```

Problem: All permutations of the same characters produce the same hash. The order of characters is completely ignored. Words like "listen" and "silent" would collide, and so would "stop" and "post."

This is a bad hash function for strings -- too many unrelated strings map to the same value.

A Better Approach: Polynomial Hashing

To make the hash depend on the **position** of each character, multiply each character by a different power of a constant p:

$$h(s) = \left(s[0] \cdot p^0 + s[1] \cdot p^1 + s[2] \cdot p^2 + \ldots + s[n-1] \cdot p^{n-1}\right) \mod m$$

Common choices: $p = 31$ or $p = 37$ (small primes).

Example with p = 31, m = 1000:

```
h("abc") = (97 * 1  +  98 * 31  +  99 * 961) mod 1000
         = (97 + 3038 + 95139) mod 1000
         = 98274 mod 1000
         = 274

h("bca") = (98 * 1  +  99 * 31  +  97 * 961) mod 1000
         = (98 + 3069 + 93217) mod 1000
         = 96384 mod 1000
         = 384
```

Now "abc" and "bca" hash to **different slots** (274 vs 384). Order matters.

This is how real languages do it. Java's `String.hashCode()` uses polynomial hashing with $p = 31$. Python uses a similar scheme with additional randomization for security.

Part 3: When Hash Functions Fail -- Collisions

What is a Collision?

A **collision** occurs when two distinct keys produce the same hash value:

$$k_1 \neq k_2 \quad \text{but} \quad h(k_1) = h(k_2)$$

Example: Table size $m = 10$, hash function $h(k) = k \mod 10$

```
h(25) = 5
h(35) = 5      <-- Collision! Both keys want slot 5.
h(45) = 5      <-- Another collision!
```

We have one slot but three keys that want to live there. Who gets the slot? What happens to the others?

This is the central problem of hashing. No matter how clever your hash function is, collisions **will** happen. They are not a bug -- they are an unavoidable mathematical reality.

The Pigeonhole Principle: Why Collisions Are Inevitable

The **Pigeonhole Principle** states:

> If you have n items and m containers, and $n > m$, then at least one container must hold more than one item.

Applied to hashing:

- **Items** = keys to store
- **Containers** = slots in the hash table

If we have a hash table of size $m = 10$ and try to store $n = 11$ keys, at least **two keys must share a slot** -- guaranteed, no matter how good the hash function is.

```
10 slots, 11 keys:

Slot:   0   1   2   3   4   5   6   7   8   9
Keys:  [k1] [k2] [k3] [k4] [k5] [k6,k7] [k8] [k9] [k10] [k11]
                                    ^
                          At least one collision must occur
```

Even with fewer keys than slots, collisions can still happen by chance. If you store 5 keys in a table of 10 slots, there is no guarantee that all 5 land in different slots.

Takeaway: Collisions are not a flaw in the hash function. They are a mathematical certainty that every hash table must handle gracefully.

How Likely Are Collisions? -- The Birthday Paradox

The **Birthday Paradox** (also called the Birthday Problem) provides a surprising answer:

> In a room of just 23 people, there is a greater than 50% chance that two people share the same birthday.

With 365 days in a year and only 23 people, a collision seems unlikely -- but it is not. The number of possible **pairs** grows rapidly: 23 people give us $23 \times 22/2 = 253$ pairs to check.

How this applies to hashing:

For a hash table of size m, if we insert approximately $\sqrt{m}$ random keys, there is a roughly 50% chance of at least one collision.

Table Size (m)	Expected Collision After	That Is Only...
100	~12 keys	12% full
1,000	~39 keys	4% full
1,000,000	~1,177 keys	0.1% full

Even a table that is only 0.1% full can have collisions. This drives home the point: **collision handling is not optional -- it is essential.**

Two Families of Collision Resolution

Since collisions are inevitable, every hash table must have a strategy for handling them. There are two main approaches:

1. Separate Chaining (Open Hashing)

Each slot holds a **linked list** (or another secondary structure) of all elements that hash to that slot. Multiple keys can coexist peacefully at the same address.

```
Slot 0: [35] --> NULL
Slot 1: [15] --> [8] --> [22] --> NULL
Slot 2: NULL
```

2. Open Addressing (Closed Hashing)

All elements are stored **inside the table itself**. If a slot is occupied, we **probe** other slots in a deterministic sequence until we find an empty one.

```
Slot 5 occupied? Try slot 6. Occupied? Try slot 7. Empty! Place here.
```

We will explore both methods in complete detail in the **next lecture**. For now, know that both approaches have distinct strengths and trade-offs.

A Brief Word: Universal Hashing

What if an adversary knows your hash function and deliberately chooses keys that all hash to the same slot? Every operation would degrade to O(n).

Universal hashing defends against this. Instead of using a single fixed hash function, you **randomly select** a hash function from a carefully designed family $\mathcal{H}$ at the start of the program.

A family $\mathcal{H}$ is called **universal** if, for any two distinct keys x and y:

$$\Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq \frac{1}{m}$$

Since the adversary does not know which function was chosen, they cannot engineer worst-case inputs. No matter what keys they pick, the **expected** number of collisions remains low.

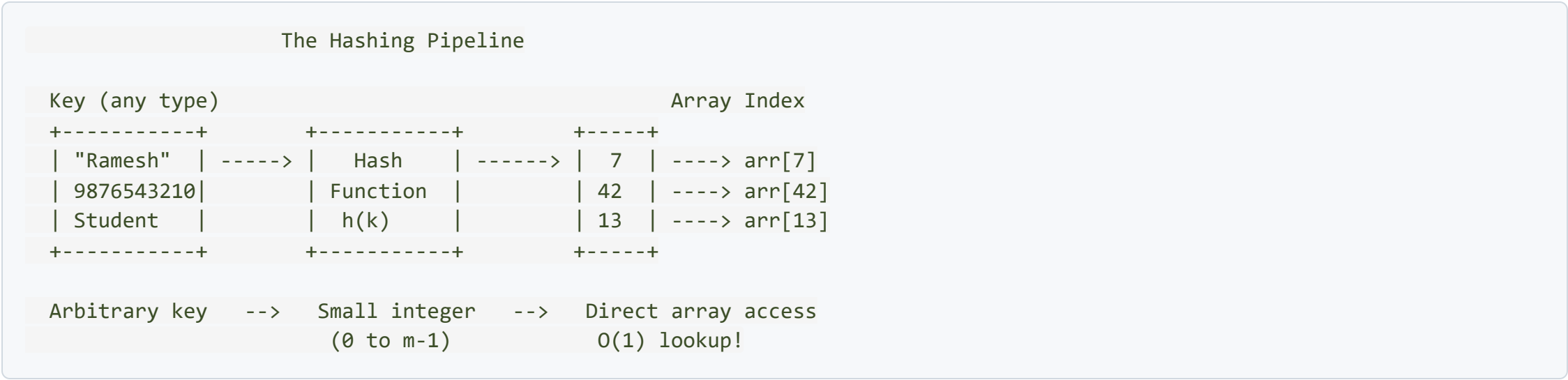
Real-world usage:

- Python (since version 3.3) randomizes hash seeds at startup to prevent hash-flooding attacks
- Java introduced alternative hashing in `HashMap` for the same reason
- Cryptographic hash tables in security-critical systems always use universal or randomized hashing

Part 4: Putting It All Together

The Big Picture So Far

We started with a simple question: can we do better than O(log n) for search? The answer is **yes**, using hashing.



What we have covered today:

Topic	Key Idea
Direct Addressing	Use key as index -- perfect but impractical for large/non-integer keys
Hash Functions	Map large key space to small index space
Division Method	$h(k) = k \mod m$, choose m as a prime
Multiplication Method	$h(k) = \lfloor m(kA \mod 1) \rfloor$, works for any m
String Hashing	Polynomial hashing gives position-dependent hashes
Collisions	Inevitable (Pigeonhole Principle, Birthday Paradox)

The Division Method vs The Multiplication Method

Let us compare both methods with a concrete example using the same set of keys and a table of size $m = 7$:

Keys: 15, 29, 42, 68, 101

Division Method: $h(k) = k \mod 7$

Key	Computation	Slot
15	15 mod 7 = 1	1
29	29 mod 7 = 1	1 (collision with 15!)
42	42 mod 7 = 0	0
68	68 mod 7 = 5	5
101	101 mod 7 = 3	3

Multiplication Method: $h(k) = \lfloor 7 \cdot (k \cdot 0.618 \mod 1) \rfloor$

Key	$k \times 0.618$	Fractional	$\times 7$	Slot
15	9.270	0.270	1.890	1
29	17.922	0.922	6.454	6
42	25.956	0.956	6.692	6 (collision with 29!)
68	42.024	0.024	0.168	0
101	62.418	0.418	2.926	2

Both methods produce one collision, but with **different key pairs**. Neither method is collision-free -- both are doing their best to distribute keys evenly.

Interactive Visualization Tool

Hashing can be difficult to visualize from diagrams alone. The following tool lets you build hash tables interactively, see where keys land, and watch collisions happen in real time.

Hash Table Visualization (University of San Francisco)

Separate Chaining:
https://www.cs.usfca.edu/~galles/visualization/OpenHash.html

Open Addressing (for preview):
https://www.cs.usfca.edu/~galles/visualization/ClosedHash.html

When you visit these tools, you can:

- Choose a table size
- Insert keys one at a time
- Watch the hash function compute the slot
- See how collisions form chains (chaining) or trigger probing (open addressing)

This is a tool we will return to in the next lecture when we dive into collision resolution in detail. For now, try inserting a few keys and observe what happens when two keys land on the same slot.

Comparing Search Strategies -- Where Does Hashing Fit?

Structure	Search	Insert	Delete	Ordered?	Notes
Unsorted Array	O(n)	O(1)	O(n)	No	Simplest structure
Sorted Array	O(log n)	O(n)	O(n)	Yes	Binary search, costly insert
BST (balanced)	O(log n)	O(log n)	O(log n)	Yes	All ops logarithmic
Hash Table	**O(1) avg**	**O(1) avg**	**O(1) avg**	**No**	Fastest for lookup

The hash table wins on speed for search, insert, and delete -- but it pays a price: **no ordering**. You cannot ask a hash table for the "next larger key" or "give me all keys in sorted order" efficiently.

When to use what:

- **Need sorted traversal or range queries?** Use a BST (AVL, Red-Black, B-Tree)
- **Need only fast lookup, insert, delete?** Use a hash table
- **Need both?** Use a balanced BST (or a combination -- many databases use B-Trees for sorted access and hash indexes for equality checks)

Key Takeaways

What We Covered Today

1. The Direct Access Problem

- O(1) lookup is possible if the key is the index, but this fails for large or non-integer keys
- Solution: use a hash function to map arbitrary keys to small integer indices

2. Hash Function Design

- **Division method:** $h(k) = k \mod m$ -- simple, but m must be a prime not near a power of 2
- **Multiplication method:** $h(k) = \lfloor m(kA \mod 1) \rfloor$ -- works well for any m, uses Knuth's golden ratio constant
- **String hashing:** Polynomial hashing weights each character by its position to make the hash order-dependent

3. Collisions Are Inevitable

- The Pigeonhole Principle guarantees that collisions must occur when the number of keys exceeds the table size
- The Birthday Paradox shows collisions are likely even when the table is mostly empty
- Two families of solutions exist: separate chaining and open addressing

4. Universal Hashing

- Randomized hash function selection prevents adversarial worst-case inputs
- Used in Python, Java, and security-critical applications

Review Questions

1. A hash table has $m = 13$ slots and uses the division method. Compute the hash values for the keys: 26, 42, 71, 13, 55, 39. Do any collisions occur?
2. Using the multiplication method with $A = 0.618$ and $m = 8$, compute the hash values for the keys: 100, 200, 300, 400. Show your work for each step.
3. Compute the polynomial hash (with $p = 31$, $m = 100$) for the strings "cat" and "act." Are the hash values different? Why is that important?
4. A university has 500 students. The IT department creates a hash table of size 500 to store student records. A junior developer says: "Since we have exactly 500 students and 500 slots, there will be no collisions." Explain why this reasoning is wrong.
5. Explain why $m = 128$ is a poor choice for the division method, even though it is a convenient power of 2. What would you choose instead for a table of roughly that size?
6. A website hashes user passwords using a fixed, publicly known hash function. An attacker discovers this. Explain how universal hashing could mitigate the risk, and why Python 3.3+ randomizes hash seeds.

What Comes Next

In the **next lecture**, we will build the complete **hash table data structure** and tackle the problem we introduced today: collision resolution.

We will study two strategies in depth:

- **Separate Chaining:** Each slot holds a linked list of all elements that hash there. Simple, flexible, and forgiving.
- **Open Addressing:** All elements live inside the table array. When a collision occurs, we *probe* for an empty slot using one of three strategies:
  - Linear Probing (simple but causes clustering)
  - Quadratic Probing (reduces clustering)
  - Double Hashing (eliminates clustering entirely)

We will trace complete insertion and search examples for all methods, compare their strengths and weaknesses, and understand the critical concept of **load factor** -- the number that tells you when your hash table is getting too crowded.